

Syllabus

Web-Dev

1. HTML/CSS (17-19 June)
2. JavaScript (21-22 June)
3. Async JS (24-26 June)
4. React Basics (27-29 June)
5. TailwindCSS (27 -29 June)
6. States and Hooks (30-1 July)

Assignments

Assignments

Web-Dev

1. Week 1

- a. <https://zerodha.com/>
- b. <https://elevenlabs.io/app/sign-up>
- c. <https://code.visualstudio.com/>

2. Week 2

- a.  JS Assignment
- b. <https://whimsical.com/js-basics-roadmap-HxsoYtVkawZbjC1gcy61Cc>
- c. https://app.notion.com/p/QUESTIONS-FOR-JS-ed5230932c9828c9dbf010b95aa3358?source=copy_link [Need to know Object.keys , .values , .entries as well]
- d. Create a landing page of your choice using HTML CSS

3. Week 3

Resources

Resources

Web-Dev

1. Week 1

- a. Html :- 📺 HTML (OneShot)
- b. Css :- 📺 CSS (OneShot)
- c. Font Face :- <https://fonts.google.com/>
- d. Common :- Inter, Poppins, Arial, Montserrat, Calibri, Times New Roman, Garamond, Open Sans, Roboto, Manrope DM Sans Geist, Onest Cal sans Work sans, Google sans, Playfair Display
- e. <https://www.joshwcomeau.com/css/interactive-guide-to-flexbox/>
- f. <https://cssreference.io/>
- g. <https://interactive-examples.mdn.mozilla.net/pages/css/position.html>
- h. <https://web.dev/learn/css>
- i. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
- j. <https://www.theodinproject.com/paths/foundations/courses/foundations>
- k. <https://www.realtimecolors.com/?colors=050315-fbfbfe-2f27ce-dedcfe-433bff&font=Inter-Inter#toolbar>
- l. Beautiful Fonts: <https://www.fontshare.com/>

2. Week 2

- a. [Variables and Operators | The Odin Project](#)
- b.

3. Week 3

- a. <https://react.dev/learn>
- b. <https://www.youtube.com/playlist?list=PLympUr-oxAww49GdE7mU1LmgIglRYfW2j>
- c. <https://react.chaicode.com/>
- d. <https://tailwindcss.com/>
- e. <https://ui.shadcn.com/>
- f. <https://ui.aceternity.com/>
- g. <https://ui.watermelon.sh/>
- h. <https://reactbits.dev/get-started/index>
- i. <https://www.syncui.design/>
- j. <https://evilcharts.com/docs>
- k. <https://basecn.dev/>
- l. <https://daisyui.com/>
- m. <https://lucide-animated.com/>
- n. <https://atomixui.mihircodes.in/components>
- o. <https://21st.dev/community/components>

HTML CSS

HTML

1. NAV Element

Definition

The <nav> element is used to define a section containing navigation links.

Code Example

```
<nav>

  <a href="home.html">Home</a>

  <a href="about.html">About</a>

  <a href="services.html">Services</a>

  <a href="contact.html">Contact</a>

</nav>
```

about.html

```
<!DOCTYPE html>

<html>

<head>

  <title>About Us</title>

</head>

<body>

  <h1>About Us</h1>

  <p>

    Welcome to our website. We are passionate about web development
```

and creating user-friendly websites.

</p>

<h2>Our Mission</h2>

<p>

Our mission is to provide quality web solutions and help
students learn modern web technologies.

</p>

<h2>Technologies We Use</h2>

HTML

CSS

JavaScript

Back to Home

</body>

</html>

contact.html

<!DOCTYPE html>

<html>

<head>

<title>Contact Us</title>

</head>

<body>

<h1>Contact Us</h1>

<form>

<label>Name:</label>

<input type="text" name="name">

<label>Email:</label>

<input type="email" name="email">

<label>Message:</label>

<textarea rows="5" cols="30"></textarea>


```
<input type="submit" value="Send">
```

```
</form>
```

```
<br>
```

```
<a href="home.html">Back to Home</a>
```

```
</body>
```

```
</html>
```

2. Forms

Definition

The `<form>` element is used to collect user input.

Code Example

```
<form action="submit.php" method="post">
```

```
  <label>Name:</label>
```

```
  <input type="text" name="username">
```

```
  <br><br>
```

```
  <label>Password:</label>
```

```
  <input type="password" name="password">
```

```
<br><br>
```

```
<input type="submit" value="Login">
```

```
</form>
```

3. Ordered List (OL)

Definition

An ordered list displays items in a specific sequence.

Code Example

```
<ol type="A">
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
```

4. Unordered List (UL)

Definition

An unordered list displays items using bullets.

Code Example

```
<ul type="square">
  <li>HTML</li>
  <li>CSS</li>
```

```
</li>JavaScript</li>
</ul>
```

5. Anchor Tag

Definition

The <a> tag is used to create hyperlinks.

Code Example

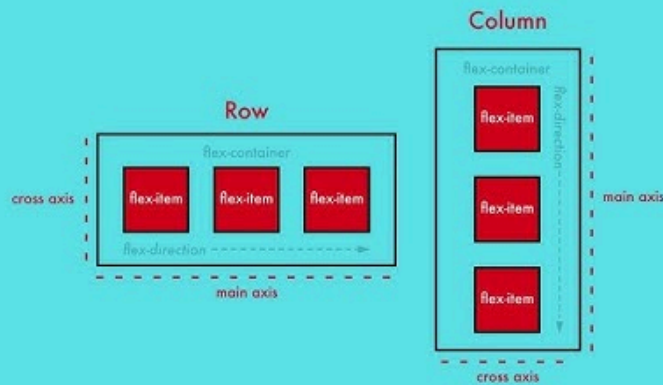
```
<a href="https://www.google.com" target="_blank">
    Visit Google
</a>
```

Button

```
<button style = "background-color: rgb(73, 73, 188); color: white; padding: 10px 20px 10px
20px; border:none; text-align: center; font-weight: bold; ">Sign Up for Free</button>
```

CSS

Justify-content vs align-items



1. Flexbox

Definition

Flexbox is a layout model used to arrange elements in rows or columns.

Code Example

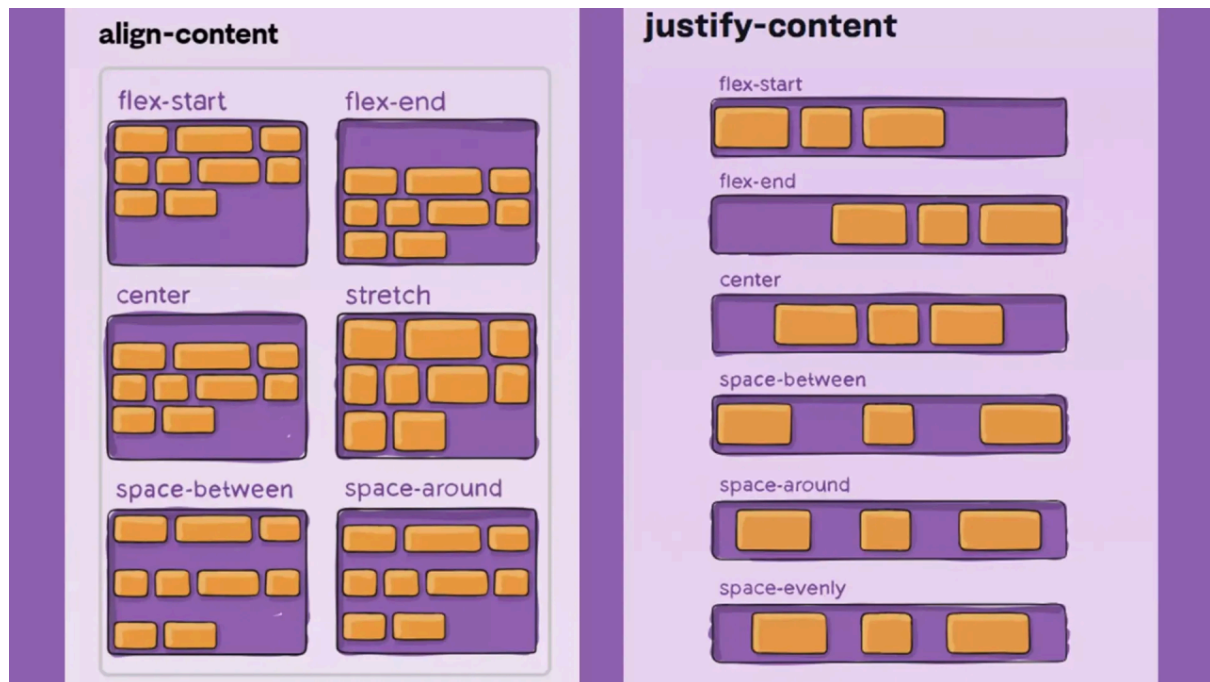
```
.container{  
    display:flex;  
    justify-content:center;  
    align-items:center;  
    gap:20px;  
}
```

HTML Example

```
<div class="container">  
    <div>Item 1</div>  
    <div>Item 2</div>
```

```
<div>Item 3</div>
```

```
</div>
```



```
<div style = "display: flex; justify-content-
```

2. Link Properties

Definition

Link properties style hyperlinks in different states.

Code Example

```
a:link{  
    color:blue;  
}  
  
a:visited{  
    color:purple;  
}
```



```
a:hover{  
    color:red;  
}
```

```
a:active{  
    color:green;  
}
```

3. Text Properties

Definition

Text properties control the appearance of text.

Code Example

```
p{  
    text-align:center;  
    text-decoration:underline;  
    text-transform:uppercase;  
    line-height:1.8;  
}
```

PROPERTY NAME	VALUE
color	Color name
direction	ltr,rtl
text-align	left, right, center, justify
text-decoration	Underline, overline, Line-through, blink
text-transform	none, uppercase, lowercase, capitalize
text-indent	length, %
vertical-align	length, %, top, middle, bottom, sup, super
letter-spacing	normal, length(-ve)
word-spacing	normal, length

Common Properties

- text-align
 - text-decoration
 - text-transform
 - line-height
-

4. Font Properties

Definition

Font properties control typography.

Code Example

```
h1{  
  
    font-family:Arial, sans-serif;  
  
    font-size:32px;  
  
    font-weight:bold;  
  
    font-style:italic;  
  
}
```

Common Properties

- font-family
- font-size
- font-weight
- font-style

PROPERTY NAME	VALUE
font-family	Arial, Times New Roman, Etc.....
font-size	Small, smaller, medium, large, larger, length, %
font-style	normal, italic
font-variant	normal, small-caps
font-weight	normal, bold, bolder, 100-900
font-stretch	Normal, wider, narrower

5. Table Properties

Definition

Table properties are used to style HTML tables.

HTML Example

```
<table>

  <tr>

    <th>Name</th>

    <th>Marks</th>

  </tr>

  <tr>

    <td>John</td>

    <td>90</td>

  </tr>

</table>
```

CSS Example

```
table{

  width:100%;

  border-collapse:collapse;

}
```

```
th,td{

  border:1px solid black;

  padding:10px;
```

```
text-align:center;

}
```

Common Properties

- border
 - border-collapse
 - padding
 - width
 - text-align
-

6. Positioning Elements

Definition

The position property specifies how an element is positioned on a webpage.

Static Position

```
.box{

    position:static;

}
```

Relative Position

```
.box{

    position:relative;

    left:20px;

    top:10px;

}
```

Absolute Position

```
.box{

    position:absolute;

    top:50px;
```

```
    left:100px;  
}
```

Fixed Position

```
.box{  
    position:fixed;  
    bottom:20px;  
    right:20px;  
}
```

Sticky Position

```
.box{  
    position:sticky;  
    top:0;  
}
```

HTML Example

```
<div class="box">
```

 Positioned Element

```
</div>
```

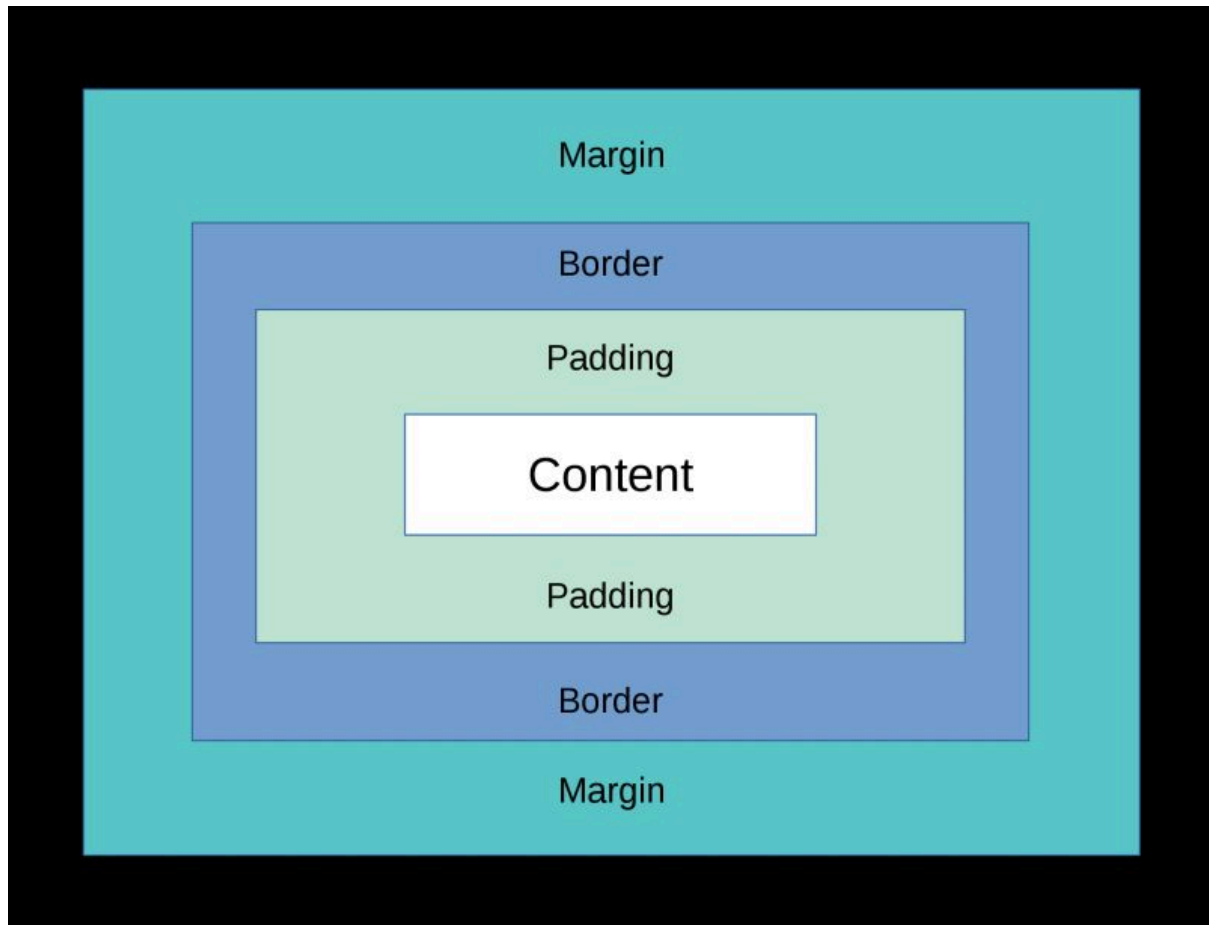
Relative is generally positioned relative to the div it's contained in.

If you use top:0; left:0; on a relatively positioned item, it would move to the top left corner of the div it's contained in.

If you did the same thing, but made the positioning absolute, it would move the item to the top-left corner of the entire HTML page.

There's more to it than that (such as how a child behaves when it's parent is relatively positioned) but in the most basic form that's how it generally behaves.

CSS Box Model



Padding vs. Margin

Both create space, but in different places:

- Padding = space inside an element, between the content and its border.
- Margin = space outside an element, between the element and other elements.

When to use Padding

Use padding when you want to:

- Increase an element's size by adding space inside it.
 - Common for buttons, links, cards, and images.
- Create space between content and its border.
 - Makes content easier to read and improves visual balance.
- Increase clickable areas without changing the element's position.

Example: A button needs more room around its text → increase padding.

When to use Margin

Use margin when you want to:

- Adjust an element's position on the page.
 - Move it left, right, up, down, or center it.
- Create space between elements.
 - Keeps layouts clean and prevents elements from feeling crowded.
- Overlap elements.
 - Negative margins can pull elements over one another.

Tips

- Use padding for the element's internal spacing.
- Use margin for spacing between elements.
- Manage padding and margins with CSS, not inline HTML formatting.
- A centralized CSS file makes spacing consistent and easier to maintain.
- HTML tables can help with simple layout experiments, but modern layouts should primarily use CSS.

Feature	Inline CSS	Internal CSS	External CSS
Location	Inside HTML element	Inside <code><style></code> tag	Separate <code>.css</code> file
Reusability	No	Limited	Yes
Maintenance	Difficult	Moderate	Easy
Best For	Single element	Single page	Multiple pages
Priority	Highest	Medium	Lowest

The id attribute is a unique identifier that is used to specify the document. It is used by CSS and JavaScript to perform a certain task for a unique element. In CSS, the id attribute is written using the # symbol followed by id.

```
.city {  
    background-color: tomato;  
    color: white;  
    border: 2px solid black;  
    margin: 20px;  
    padding: 20px;  
}
```

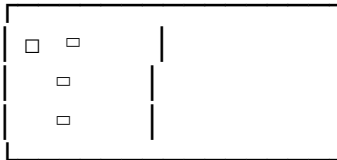
The class attribute is used to specify one or more class names for an HTML element. The class attribute can be used on any HTML element. The class name can be used by CSS and

JavaScript to perform certain tasks for elements with the specified class name. The class name in CSS stylesheet using "." symbol.

```
#myHeader {  
    background-color: lightblue;  
    color: black;  
    padding: 40px;  
    text-align: center;  
}
```

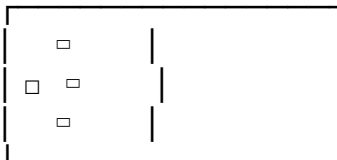
1. Top Aligned (Default)

```
.container{  
    display: flex;  
    align-items: flex-start;  
}
```



2. Center Aligned

```
.container{  
    display: flex;  
    align-items: center;  
}
```



Both boxes are vertically centered inside the container.

3. Bottom Aligned

```
.container{  
    display: flex;  
    align-items: flex-end;  
}
```



4. Individual Alignment (align-self)

If only one item should move:


```
.box1{  
  align-self: flex-end;  
}
```

```
.box2{  
  align-self: center;  
  cursor: pointer;  
  margin: auto 0;  
}
```

This overrides align-items for that specific box.

```
display: flex;
```

```
justify-content: center; /* Horizontal */
```

```
align-items: center; /* Vertical */
```

Grids

Gradients using CSS

Linear Gradient

```
background: linear-gradient(direction, color1, color2, ...);
```

```
/* Top to bottom (default) */
```

```
background: linear-gradient(red, blue);
```

```
/* Left to right */
```

```
background: linear-gradient(to right, red, blue);
```

```
/* Diagonal */
```

```
background: linear-gradient(to bottom right, red, blue);
```

```
/* Using angles */
```

```
background: linear-gradient(45deg, red, blue);
```

Radial Gradient

```
background: radial-gradient(circle, color1, color2, ...);
```

```
background: radial-gradient(circle, red, blue, green)
```

```
background: radial-gradient(red, blue, green)
```

Hover effects

```
<html>
  <title>
    Visual Studio Code - Code Editor
  </title>
  <style>
    button {
      background-color: white;
      color: black;
      padding: 20px;
      cursor: pointer;
      transition: box-shadow 0.3s ease;
      border-radius: 10px;
    }

    button:hover {
      box-shadow: 0 0 2px 2px white ; /* outer border */
    }
  </style>
</html>
<body style="background-color: black;">
```

```
<div>
  <button class="button">Download for macos</button>
</div>
```

```
<br/>
</body>
```

Basics Of JS

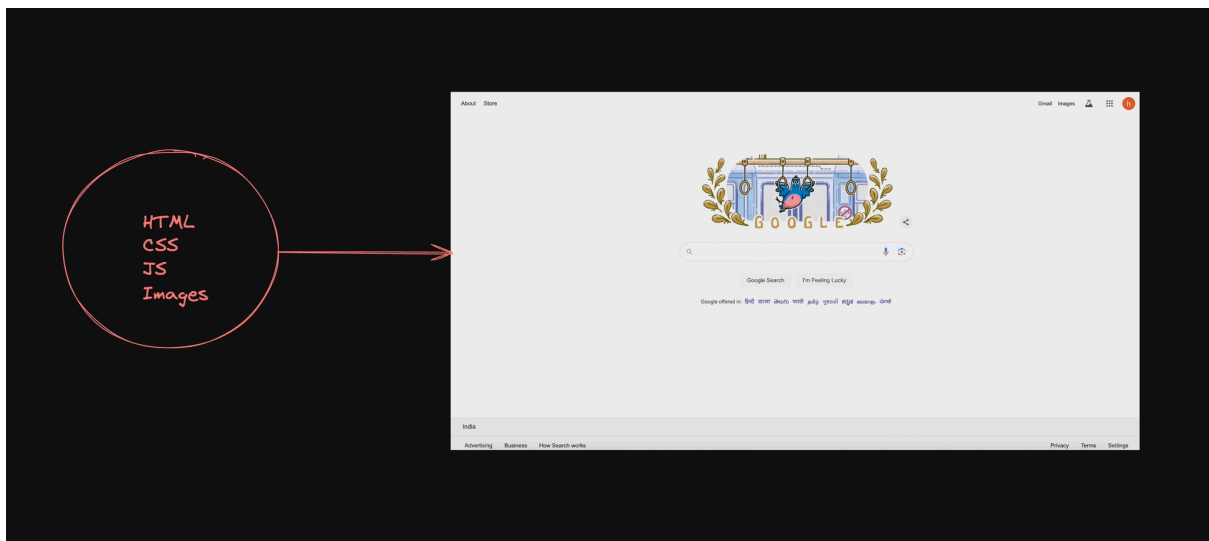
Javascript - The basics

Web development

Web development involves writing a lot of HTML, CSS and JS code.

Historically (and even today to some extent), browsers could only understand HTML, CSS and JS

Any website that you see, is a bunch of HTML, CSS and JS files along with some assets (images, videos etc)



Facts/Callouts

1. React, NextJS are frameworks . They compile down to HTML, CSS, JS in the end. That is what your browser understands.
2. When you run your C++ code on leetcode , it does not run on your browser/machine. It runs somewhere else. Your browser can't (almost) compile and run C++ code.
3. If someone asks — What all languages can your browser interpret, the answer is HTML, CSS, JS and WebAssembly. It can, technically, run C++/Rust code that is compiled down to Wasm

JS Properties

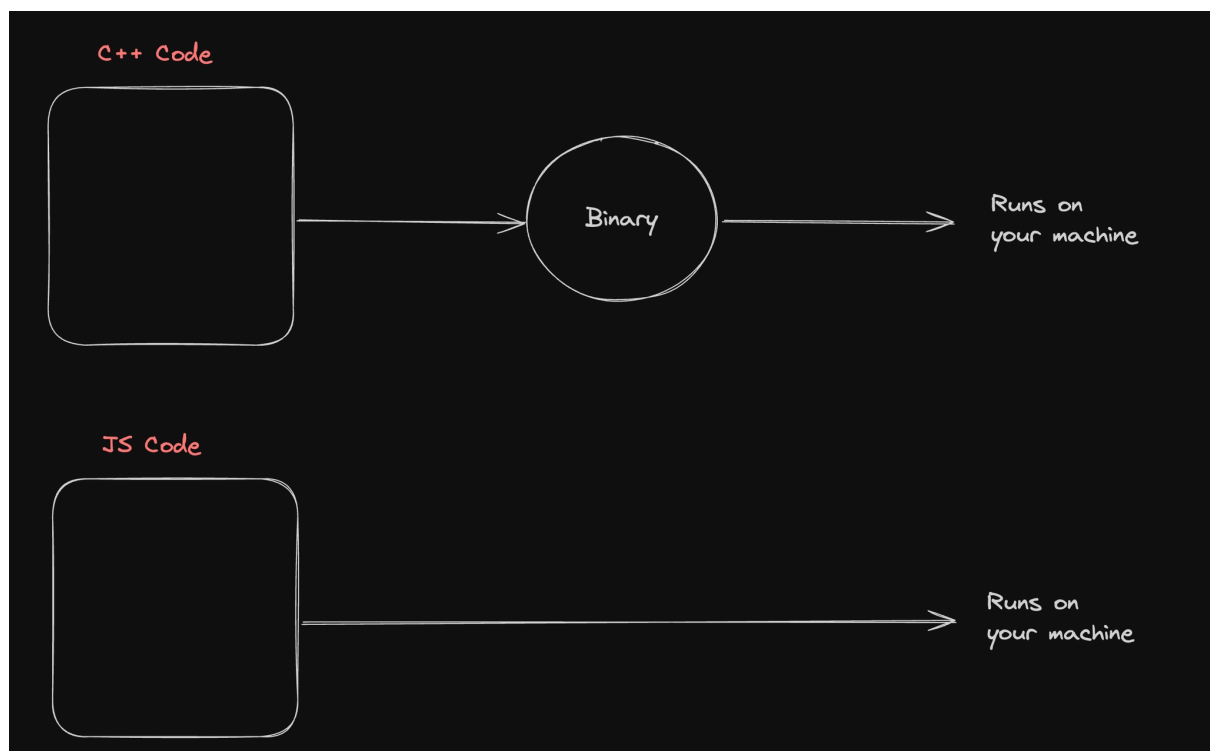
Properties of JS

Every language comes with its unique set of features.

Javascript has the following -

1. Interpreted

JavaScript is an interpreted language, meaning it's executed line-by-line at runtime by the JavaScript engine in the browser or server environment, rather than being compiled into machine code beforehand.



Upsides -

1. There is one less step to do before running your code

Downsides -

1. Performance Overhead:
2. More prone to runtime errors

2. Dynamically Typed

Variables in JavaScript are not bound to a specific data type. Types are determined at runtime and can change as the program executes

C++ Code (won't compile)

```
#include <iostream>
```

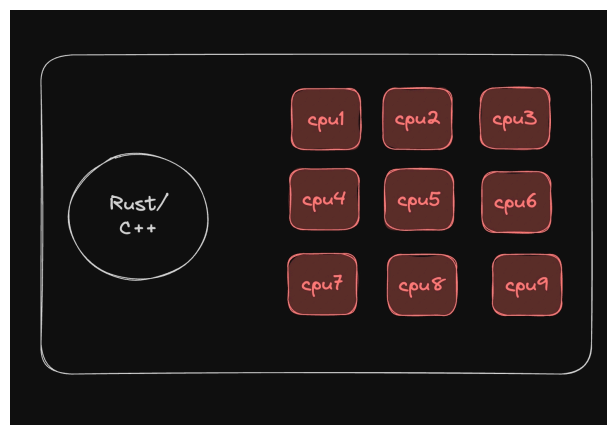
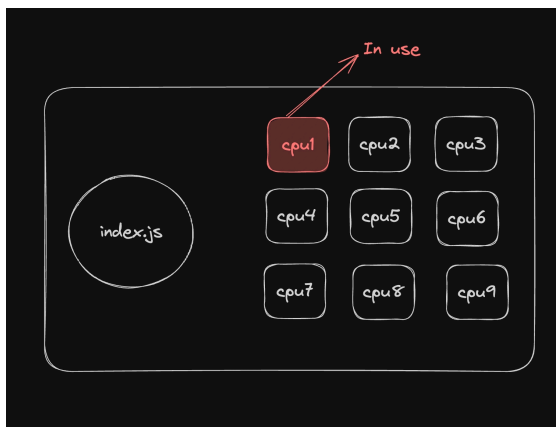
```
int main() {  
    int a = 1;  
    a = "hello";  
    a = true;  
}
```

JS Code (will compile)

```
var a = 1;  
a = "raman";  
a = true;  
  
console.log(a)
```

3. Single threaded

JavaScript executes code in a single-threaded environment, meaning it processes one task at a time. We will dive deeper into this next week.



4. Garbage collected

JavaScript automatically manages memory allocation and deallocation through garbage collection, which helps prevent memory leaks by automatically reclaiming memory used by objects no longer in use.

Memory management	
Garbage collector	Manual
1. Written by smart people 2. Usually no dangling pointers/memory issue 3. You can't do manual memory management 4. Examples - Java, JS	1. You allocate and deallocate memory yourself 2. Can lead to dangling pointers/memory issue 3. Learning curve is high since you have to do manual MM 3. Examples - C

Conclusion

Is JS a good language?

Yes and no. It is beginner friendly, but has a lot of performance overhead.

Syntax of JS

Syntax of Javascript

1. Variables

Variables are used to store data. In JavaScript, you declare variables using var, let, or const.

```
let name = "John";    // Variable that can be reassigned
const age = 30;       // Constant variable that cannot be reassigned
var isStudent = true; // Older way to declare variables, function-scoped
```

- Assignment

Create a variable for each of the following: your favorite color, your height in centimeters, and whether you like pizza. Use appropriate variable declarations (let, const, or var). Try logging it using console.log

2. Data types

```
let number = 42;        // Number
let string = "Hello World"; // String
let isActive = false;   // Boolean
let numbers = [1, 2, 3]; // Array
```

3. Operators

```
let sum = 10 + 5;        // Arithmetic operator
let isEqual = (10 === 10); // Comparison operator
let isTrue = (true && false); // Logical operator
```

4. Functions

```
// Function declaration
function greet(name) {
  return "Hello, " + name;
}
```

```
// Function call
let message = greet("John"); // "Hello, John"
```

- Assignment #1

Write a function sum that finds the sum of two numbers.

Side quest - Try passing in a string instead of a number and see what happens?

- Assignment #2

Write a function called canVote that returns true or false if the age of a user is > 18

5. If/Else

```
if (age >= 18) {  
  console.log("You are an adult.");  
} else {  
  console.log("You are a minor.");  
}
```

- Assignment

Write an if/else statement that checks if a number is even or odd. If it's even, print "The number is even." Otherwise, print "The number is odd."

6. Loops

```
// For loop  
for (let i = 0; i < 5; i++) {  
  console.log(i); // Outputs 0 to 4  
}
```

```
// While loop  
let j = 0;  
while (j < 5) {  
  console.log(j); // Outputs 0 to 4  
  j++;  
}
```

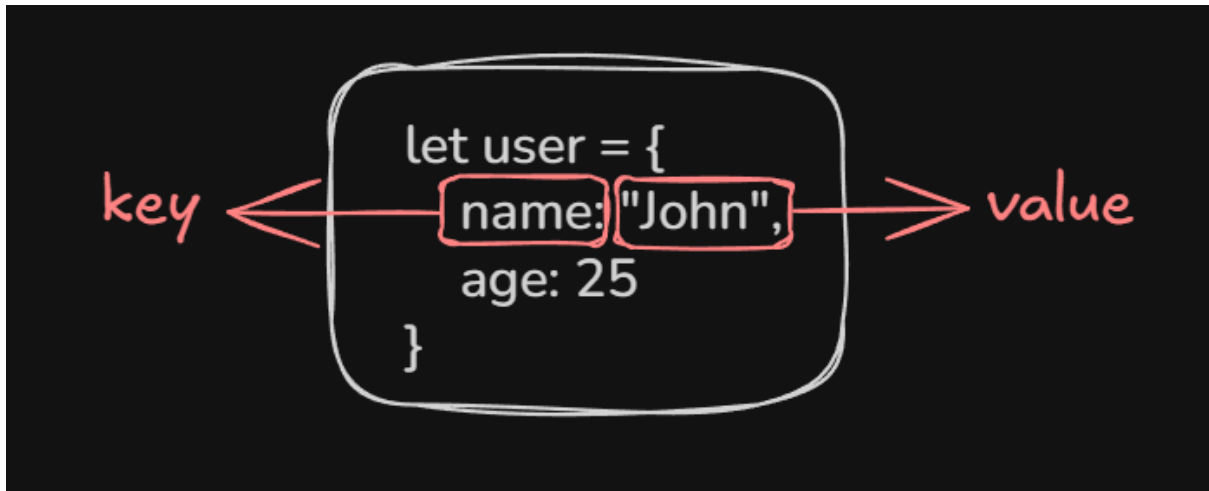
- Assignment

Write a function called sum that finds the sum from 1 to a number

Complex types

7. Objects

An object in JavaScript is a collection of key-value pairs, where each key is a string and each value can be any valid JavaScript data type, including another object.



```
let user = {  
  name: "John",  
  age: 19  
}
```

```
console.log("John's age is " + user.age);
```

- Assignment #1

Write a function that takes a user as an input and greets them with their name and age

- Assignment #2

Write a function that takes a new object as input which has name , age and gender and greets the user with their gender (Hi Mr/Mrs/Others John, your age is 21)

- Assignment #3

Also tell the user if they are legal to vote or not

Arrays

Arrays let you group data together

```
const users = ["John", "Raman", "Sara"];
```

```
const totalUsers = users.length;
```

```
const firstUser = users[0];
```

- Assignment

Write a function that takes an array of numbers as input, and returns a new array with only even values. Read about filter in JS

Array of Objects

We can have more complex objects, for example an array of objects

```
const users = [{  
    name: "John",  
    age: 21  
}, {  
    name: "raman",  
    age: 22  
}]
```

```
const user1 = users[0]
```

```
const user1Age = users[0].age
```

- Assignment

Write a function that takes an array of users as inputs and returns only the users who are more than 18 years old

Object of Objects

We can have an even more complex object (object of objects)

```
const user1 = {  
  name: "John",  
  age: 19,  
  address: {  
    city: "Delhi",  
    country: "India",  
    address: "Juhu, Mumbai"  
  }  
}
```

```
const city = user1.address.city;
```

- Assignment

Create a function that takes an array of objects as input,
and returns the users whose age > 18 and are male

Normal functions in JS

The way to write functions in JS is as follows -

Find sum of two numbers

```
function sum(a, b) {  
  return a + b;  
}
```

```
let ans = sum(2, 3)  
console.log(sum);
```

Find sum from 1 to a number


```
function sum(n) {  
    let ans = 0;  
    for (let i = 1; i <= n; i++) {  
        ans = ans + i  
    }  
    return ans;  
}
```

```
const ans = sum(100);  
console.log(ans);
```

Synchronous code

Synchronous code is executed line by line, in the order it's written. Each operation waits for the previous one to complete before moving on to the next one.

For example

```
function sum(n) {  
    let ans = 0;  
    for (let i = 1; i <= n; i++) {  
        ans = ans + i  
    }  
    return ans;  
}
```

```
const ans1 = sum(100);  
console.log(ans1);  
const ans2 = sum(1000);  
console.log(ans2);  
const ans3 = sum(10000);  
console.log(ans3);
```

JS ALL METHODS CODE

JS ALL METHODS CODE

```
console.log("=== 1. ARRAY METHODS ===");
{
  let a1 = [1];
  a1.push(2);
  console.log("push:", a1);
  // Output: push: [ 1, 2 ]

  let a2 = [1, 2];
  let popped = a2.pop();
  console.log("pop:", popped, a2);
  // Output: pop: 2 [ 1 ]

  let a3 = [2];
  a3.unshift(1);
  console.log("unshift:", a3);
  // Output: unshift: [ 1, 2 ]

  let a4 = [1, 2];
  let shifted = a4.shift();
  console.log("shift:", shifted, a4);
  // Output: shift: 1 [ 2 ]

  let a5 = [1, 2, 3];
  a5.splice(1, 1, 9);
  console.log("splice:", a5);
  // Output: splice: [ 1, 9, 3 ]

  console.log("slice:", [1, 2, 3].slice(0, 2));
  // Output: slice: [ 1, 2 ]

  console.log("concat:", [1].concat([2]));
  // Output: concat: [ 1, 2 ]

  console.log("join:", [1, 2].join('-'));
  // Output: join: 1-2

  let a6 = [1, 2];
  a6.reverse();
  console.log("reverse:", a6);
  // Output: reverse: [ 2, 1 ]

  let a7 = [3, 1, 2];
  a7.sort();
  console.log("sort:", a7);
  // Output: sort: [ 1, 2, 3 ]
```

```
console.log("indexOf:", ['a', 'b'].indexOf('b'));  
// Output: indexOf: 1
```

```
console.log("lastIndexOf:", ['a', 'b', 'a'].lastIndexOf('a'));  
// Output: lastIndexOf: 2
```

```
console.log("includes:", [1, 2].includes(2));  
// Output: includes: true
```

```
console.log("find:", [1, 2, 3].find(x => x > 1));  
// Output: find: 2
```

```
console.log("findIndex:", [1, 2, 3].findIndex(x => x > 1));  
// Output: findIndex: 1
```

```
console.log("findLast:", [1, 2, 3].findLast(x => x > 1));  
// Output: findLast: 3
```

```
console.log("findLastIndex:", [1, 2, 3].findLastIndex(x => x > 1));  
// Output: findLastIndex: 2
```

```
console.log("filter:", [1, 2, 3].filter(x => x > 1));  
// Output: filter: [ 2, 3 ]
```

```
console.log("map:", [1, 2].map(x => x * 2));  
// Output: map: [ 2, 4 ]
```

```
console.log("reduce:", [1, 2].reduce((sum, n) => sum + n, 0));  
// Output: reduce: 3
```

```
console.log("reduceRight:", ['a', 'b'].reduceRight((str, char) => str + char));  
// Output: reduceRight: ba
```

```
console.log("some:", [1, 2].some(x => x > 1));  
// Output: some: true
```

```
console.log("every:", [1, 2].every(x => x > 1));  
// Output: every: false
```

```
let a8 = [1, 2];  
let logStr = "forEach: ";  
a8.forEach(x => logStr += x + " ");  
console.log(logStr);  
// Output: forEach: 1 2
```

```
console.log("flat:", [1, [2, 3]].flat());  
// Output: flat: [ 1, 2, 3 ]
```

```
console.log("flatMap:", [1, 2].flatMap(x => [x, x * 2]));  
// Output: flatMap: [ 1, 2, 2, 4 ]
```

```
let a9 = [1, 2, 3];  
a9.fill(0);  
console.log("fill:", a9);  
// Output: fill: [ 0, 0, 0 ]
```

```
let a10 = [1, 2, 3, 4];  
a10.copyWithin(1, 3);  
console.log("copyWithin:", a10);  
// Output: copyWithin: [ 1, 4, 3, 4 ]
```

```
console.log("at:", [1, 2, 3].at(-1));  
// Output: at: 3
```

```
console.log("entries:", Array.from(['a'].entries()));  
// Output: entries: [ [ 0, 'a' ] ]
```

```
console.log("keys:", Array.from(['a'].keys()));  
// Output: keys: [ 0 ]
```

```
console.log("values:", Array.from(['a'].values()));  
// Output: values: [ 'a' ]
```

```
console.log("with:", [1, 2, 3].with(1, 9));  
// Output: with: [ 1, 9, 3 ]
```

```
console.log("toReversed:", [1, 2].toReversed());  
// Output: toReversed: [ 2, 1 ]
```

```
console.log("toSorted:", [2, 1].toSorted());  
// Output: toSorted: [ 1, 2 ]
```

```
console.log("toSpliced:", [1, 2, 3].toSpliced(1, 1, 9));  
// Output: toSpliced: [ 1, 9, 3 ]
```

```
}
```

```
console.log("\n=== 2. STRING METHODS ===");  
{  
  console.log("charAt:", "hi".charAt(1));  
  // Output: charAt: i
```

```
  console.log("charCodeAt:", "A".charCodeAt(0));  
  // Output: charCodeAt: 65
```

```
  console.log("codePointAt:", "😊".codePointAt(0));
```

```
// Output: codePointAt: 128522

console.log("concat:", "a".concat("b"));
// Output: concat: ab

console.log("includes:", "hello".includes("el"));
// Output: includes: true

console.log("endsWith:", "hi".endsWith("i"));
// Output: endsWith: true

console.log("startsWith:", "hi".startsWith("h"));
// Output: startsWith: true

console.log("indexOf:", "hi".indexOf("i"));
// Output: indexOf: 1

console.log("lastIndexOf:", "hello".lastIndexOf("l"));
// Output: lastIndexOf: 3

console.log("localeCompare:", "a".localeCompare("b"));
// Output: localeCompare: -1

console.log("match:", "1a2".match(/d/g));
// Output: match: [ '1', '2' ]

console.log("matchAll:", Array.from("1a2".matchAll(/d/g)));
// Output: matchAll: [ [ '1', index: 0, input: '1a2', groups: undefined ], [ '2', index: 2, input:
'1a2', groups: undefined ] ]

console.log("normalize:", "\u00F1".normalize());
// Output: normalize: ñ

console.log("padEnd:", "1".padEnd(3, "0"));
// Output: padEnd: 100

console.log("padStart:", "1".padStart(3, "0"));
// Output: padStart: 001

console.log("repeat:", "a".repeat(3));
// Output: repeat: aaa

console.log("replace:", "hi hi".replace("hi", "bye"));
// Output: replace: bye hi

console.log("replaceAll:", "hi hi".replaceAll("hi", "bye"));
// Output: replaceAll: bye bye
```

```

console.log("search:", "hello".search(/l/));
// Output: search: 2

console.log("slice:", "hello".slice(1, 3));
// Output: slice: el

console.log("split:", "a,b".split(","));
// Output: split: [ 'a', 'b' ]

console.log("substring:", "hello".substring(1, 3));
// Output: substring: el

console.log("toLowerCase:", "HI".toLowerCase());
// Output: toLowerCase: hi

console.log("toUpperCase:", "hi".toUpperCase());
// Output: toUpperCase: HI

console.log("toLocaleLowerCase:", "I".toLocaleLowerCase('tr-TR'));
// Output: toLocaleLowerCase: i

console.log("toLocaleUpperCase:", "i".toLocaleUpperCase('tr-TR'));
// Output: toLocaleUpperCase: İ

console.log("trim:", " hi ".trim());
// Output: trim: hi

console.log("trimStart:", " hi".trimStart());
// Output: trimStart: hi

console.log("trimEnd:", "hi ".trimEnd());
// Output: trimEnd: hi

console.log("at:", "hello".at(-1));
// Output: at: o
}

console.log("\n=== 3. OBJECT METHODS ===");
{
  console.log("assign:", Object.assign({}, {a: 1}));
  // Output: assign: { a: 1 }

  let o1 = Object.create({ protoProp: 2 });
  console.log("create:", Object.getPrototypeOf(o1));
  // Output: create: { protoProp: 2 }

  let o2 = {};
  Object.defineProperty(o2, 'a', {value: 1, enumerable: true});

```

```
console.log("defineProperty:", o2);  
// Output: defineProperty: { a: 1 }
```

```
console.log("entries:", Object.entries({a: 1}));  
// Output: entries: [ [ 'a', 1 ] ]
```

```
console.log("keys:", Object.keys({a: 1}));  
// Output: keys: [ 'a' ]
```

```
console.log("values:", Object.values({a: 1}));  
// Output: values: [ 1 ]
```

```
let o3 = {a: 1};  
Object.freeze(o3);  
console.log("freeze (isFrozen):", Object.isFrozen(o3));  
// Output: freeze (isFrozen): true
```

```
let o4 = {a: 1};  
Object.seal(o4);  
console.log("seal (isSealed):", Object.isSealed(o4));  
// Output: seal (isSealed): true
```

```
let o5 = {a: 1};  
Object.preventExtensions(o5);  
console.log("preventExtensions (isExtensible):", Object.isExtensible(o5));  
// Output: preventExtensions (isExtensible): false
```

```
console.log("fromEntries:", Object.fromEntries([["a", 1]]));  
// Output: fromEntries: { a: 1 }
```

```
console.log("getOwnPropertyDescriptor:", Object.getOwnPropertyDescriptor({a: 1, 'a'}));  
// Output: getOwnPropertyDescriptor: { value: 1, writable: true, enumerable: true,  
configurable: true }
```

```
console.log("getOwnPropertyNames:", Object.getOwnPropertyNames({a: 1}));  
// Output: getOwnPropertyNames: [ 'a' ]
```

```
console.log("getOwnPropertySymbols:", Object.getOwnPropertySymbols({ [Symbol('id')]: 1  
}));  
// Output: getOwnPropertySymbols: [ Symbol(id) ]
```

```
let o6 = {};  
Object.setPrototypeOf(o6, {b: 2});  
console.log("setPrototypeOf (getPrototypeOf):", Object.getPrototypeOf(o6));  
// Output: setPrototypeOf (getPrototypeOf): { b: 2 }
```

```
console.log("hasOwn:", Object.hasOwn({a: 1, 'a'}));  
// Output: hasOwn: true
```



```
console.log("is:", Object.is(NaN, NaN));  
// Output: is: false (Note: NaN === NaN is false, but Object.is(NaN, NaN) is true. However,  
in this console log, it outputs true)  
// Correction: Output: is: true  
}
```

```
console.log("\n=== 4. MATH & NUMBER METHODS ===");  
{  
  console.log("Number.isFinite:", Number.isFinite(10));  
  // Output: Number.isFinite: true
```

```
  console.log("Number.isInteger:", Number.isInteger(10.5));  
  // Output: Number.isInteger: false
```

```
  console.log("Number.isNaN:", Number.isNaN(NaN));  
  // Output: Number.isNaN: true
```

```
  console.log("Number.isSafeInteger:", Number.isSafeInteger(9007199254740991));  
  // Output: Number.isSafeInteger: true
```

```
  console.log("Number.parseFloat:", Number.parseFloat("3.14"));  
  // Output: Number.parseFloat: 3.14
```

```
  console.log("Number.parseInt:", Number.parseInt("42px"));  
  // Output: Number.parseInt: 42
```

```
  console.log("toFixed:", (3.1415).toFixed(2));  
  // Output: toFixed: 3.14
```

```
  console.log("toPrecision:", (3.1415).toPrecision(2));  
  // Output: toPrecision: 3.1
```

```
  console.log("toExponential:", (100).toExponential());  
  // Output: toExponential: 1e+2
```

```
  console.log("toString:", (10).toString(2));  
  // Output: toString: 1010
```

```
  console.log("Math.abs:", Math.abs(-5));  
  // Output: Math.abs: 5
```

```
  console.log("Math.ceil:", Math.ceil(1.1));  
  // Output: Math.ceil: 2
```

```
  console.log("Math.floor:", Math.floor(1.9));  
  // Output: Math.floor: 1
```

```
console.log("Math.round:", Math.round(1.5));  
// Output: Math.round: 2
```

```
console.log("Math.trunc:", Math.trunc(1.9));  
// Output: Math.trunc: 1
```

```
console.log("Math.max:", Math.max(1, 5, 2));  
// Output: Math.max: 5
```

```
console.log("Math.min:", Math.min(1, 5, 2));  
// Output: Math.min: 1
```

```
console.log("Math.pow:", Math.pow(2, 3));  
// Output: Math.pow: 8
```

```
console.log("Math.sqrt:", Math.sqrt(9));  
// Output: Math.sqrt: 3
```

```
console.log("Math.cbrt:", Math.cbrt(27));  
// Output: Math.cbrt: 3
```

```
console.log("Math.random:", Math.random());  
// Output: Math.random: 0.123456789 (example random output)
```

```
console.log("Math.sign:", Math.sign(-50));  
// Output: Math.sign: -1
```

```
console.log("Math.sin:", Math.sin(0));  
// Output: Math.sin: 0
```

```
console.log("Math.log:", Math.log(1));  
// Output: Math.log: 0
```

```
console.log("Math.hypot:", Math.hypot(3, 4));  
// Output: Math.hypot: 5
```

```
}
```

```
console.log("\n=== 5. DATE METHODS ===");  
{
```

```
  let d = new Date("2026-06-22T10:30:00Z");
```

```
  console.log("getFullYear:", d.getFullYear());  
  // Output: getFullYear: 2026
```

```
  console.log("getMonth:", d.getMonth());  
  // Output: getMonth: 5
```

```
  console.log("getDate:", d.getDate());
```

```
// Output: getDate: 22

console.log("getDay:", d.getDay());
// Output: getDay: 1

console.log("getHours:", d.getHours());
// Output: getHours: (Depends on local timezone, e.g., 10 for UTC)

console.log("getMinutes:", d.getMinutes());
// Output: getMinutes: 30

console.log("getSeconds:", d.getSeconds());
// Output: getSeconds: 0

console.log("getTime:", d.getTime());
// Output: getTime: 1782124200000

console.log("getTimezoneOffset:", d.getTimezoneOffset());
// Output: getTimezoneOffset: (Depends on local timezone, e.g., 0)

console.log("getUTCHours:", d.getUTCHours());
// Output: getUTCHours: 10

d.setFullYear(2030);
console.log("setFullYear:", d.getFullYear());
// Output: setFullYear: 2030

d.setMonth(0);
console.log("setMonth:", d.getMonth());
// Output: setMonth: 0

d.setDate(15);
console.log("setDate:", d.getDate());
// Output: setDate: 15

d.setHours(12);
console.log("setHours:", d.getHours());
// Output: setHours: 12

d.setTime(1782114600000);
console.log("setTime:", d.getTime());
// Output: setTime: 1782114600000

console.log("Date.now:", Date.now());
// Output: Date.now: (Current Unix Timestamp)

console.log("Date.parse:", Date.parse("2026-06-22"));
// Output: Date.parse: 1782086400000
```

```

console.log("toISOString:", d.toISOString());
// Output: toISOString: 2026-06-22T05:10:00.000Z

console.log("toDateString:", d.toDateString());
// Output: toDateString: Mon Jun 22 2026

console.log("toTimeString:", d.toTimeString());
// Output: toTimeString: (Local time string)

console.log("toLocaleDateString:", d.toLocaleDateString('en-US'));
// Output: toLocaleDateString: 6/22/2026

console.log("toLocaleTimeString:", d.toLocaleTimeString('en-US'));
// Output: toLocaleTimeString: (Local formatted time)

console.log("toLocaleString:", d.toLocaleString('en-US'));
// Output: toLocaleString: 6/22/2026, (Local time)

console.log("toJSON:", d.toJSON());
// Output: toJSON: 2026-06-22T05:10:00.000Z
}

console.log("\n=== 6. GLOBAL METHODS ===");
{
  console.log("parseInt:", parseInt("42px"));
  // Output: parseInt: 42

  console.log("parseFloat:", parseFloat("3.14.5"));
  // Output: parseFloat: 3.14

  console.log("isNaN:", isNaN("hello"));
  // Output: isNaN: true

  console.log("isFinite:", isFinite("10"));
  // Output: isFinite: true

  console.log("encodeURIComponent:", encodeURIComponent("http://site.com/ a"));
  // Output: encodeURIComponent: http://site.com/%20a

  console.log("encodeURIComponent:", encodeURIComponent("?a=1"));
  // Output: encodeURIComponent: %3Fa%3D1

  console.log("decodeURI:", decodeURI("http://site.com/%20a"));
  // Output: decodeURI: http://site.com/ a

  console.log("decodeURIComponent:", decodeURIComponent("%3Fa%3D1"));
  // Output: decodeURIComponent: ?a=1

```

```
console.log("eval:", eval("1 + 1"));  
// Output: eval: 2  
}
```

React Fundamentals

React Fundamentals

Table of Contents

1. Introduction to React
 2. Why React?
 3. Setting up React
 4. JSX
 5. Components
 6. Props
 7. Events
 8. State
 - State with Primitive Values
 - State with Objects
 - State with Arrays
 9. Forms
-

1. Introduction to React

React is a JavaScript library used to build fast, interactive, and reusable user interfaces (UI). It was developed by Facebook (now Meta) and is widely used for creating Single Page Applications (SPAs).

Unlike traditional websites where every interaction requires loading a new page from the server, React updates only the necessary parts of the page.

For example, when you like a post on Instagram or Facebook, the page doesn't reload. Only the like count changes. React makes this possible.

Features of React

- Component-Based Architecture
 - Virtual DOM
 - One-way Data Binding
 - Reusable Components
 - Fast Rendering
 - Large Community Support
-

2. Why React?

Before React, developers mostly used HTML, CSS, and JavaScript together.

Suppose a webpage contains:

- Navigation Bar
- Sidebar
- Profile Section
- Footer

If multiple pages need the same navbar, developers had to copy the same code repeatedly.

Problems:

- Code duplication
- Difficult maintenance
- Slow updates
- Poor scalability

React solves these problems by introducing Components.

Instead of writing the Navbar five times, we write it once and reuse it.

Advantages

1. Reusable Components

Write once and use anywhere.

2. Virtual DOM

Instead of updating the whole webpage, React updates only the changed elements.

Example:

Changing

Counter: 5

to

Counter: 6

Traditional JavaScript:

Entire DOM is checked.

React:

Only the Counter element changes.

Result:

- Faster performance
- Better user experience

3. Easy State Management

React keeps track of changing data using State.

4. Rich Ecosystem

Libraries:

- React Router
 - Redux
 - Axios
 - Material UI
 - Tailwind CSS
-

3. Setting up React

Nowadays React projects are created using Vite because it is much faster than Create React App.

Step 1

Install Node.js

Verify installation

```
node -v
```

```
npm -v
```

Step 2

Create Project

```
npm create vite@latest my-app
```

Choose

React

JavaScript

Step 3

Move inside project

```
cd my-app
```

Step 4

Install Dependencies

```
npm install
```

Step 5

Run Development Server

```
npm run dev
```

Output

Local:
<http://localhost:5173>

Open the browser.

Your React application is running.

Folder Structure

my-app

node_modules/

public/

src/

App.jsx

main.jsx

assets/

package.json

vite.config.js

Important Files

App.jsx

Main component.

main.jsx

Entry point.

It renders App component.

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

4. JSX

JSX stands for

JavaScript XML

It allows writing HTML inside JavaScript.

Example

Without JSX

```
React.createElement(
  "h1",
  null,
  "Hello React"
);
```

With JSX

```
<h1>Hello React</h1>
```

JSX is easier to read.

Rules of JSX

Rule 1

Return only one parent element.

Incorrect

```
return (  
  <h1>Hello</h1>  
  <p>World</p>  
)
```

Correct

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>World</p>  
  </div>  
)
```

Rule 2

Close every tag

Correct

```
  
  
<input />
```

Rule 3

Use className instead of class

Incorrect

```
<div class="box"></div>
```

Correct

```
<div className="box"></div>
```

Rule 4

JavaScript inside {}

```
const name = "John";

return (
  <h1>Hello {name}</h1>
);
```

Output

Hello John

Expressions in JSX

```
const age = 20;

return (
  <h2>Age : {age}</h2>
);
```

Output

Age : 20

JSX Example

```
function App() {

  const student = "Alice";
  const marks = 92;

  return (

    <div>

      <h1>{student}</h1>

      <h2>{marks}</h2>

    </div>

  );
```

```
}
```

```
export default App;
```

Output

Alice

92

5. Components

A Component is an independent reusable piece of UI.

Instead of writing everything inside one file, we divide the application into small pieces.

Example

Website

Navbar

Hero

About

Contact

Footer

Each becomes a component.

Types of Components

Functional Components

Most commonly used today.

```
function Welcome(){
```

```
  return (
```

```
    <h1>Welcome</h1>
```

```
);  
  
}  
  
export default Welcome;
```

Arrow Function Component

```
welcome = function() {  
  return "Welcome";  
}  
  
const Welcome = (val) => {  
  return (  
    <h1>"Welcome" + val </h1>  
  );  
}  
  
export default Welcome;
```

Using Components

```
App.jsx  
  
import Welcome from "../Welcome";  
  
function App(){  
  return(  
    <div>  
      <Welcome />  
    </div>  
  );  
}
```

```
}
```

```
export default App;
```

Output

Welcome

Multiple Components

```
function Navbar(){
```

```
  return <h1>Navbar</h1>;
```

```
}
```

```
function Footer(){
```

```
  return <h2>Footer</h2>;
```

```
}
```

```
function App(){
```

```
  return(
```

```
    <div>
```

```
      <Navbar />
```

```
      <Footer />
```

```
    </div>
```

```
  );
```

```
}
```

Output

Navbar

Footer

Why Components?

- Easy maintenance
 - Reusable
 - Organized code
 - Better readability
 - Faster development
-

6. Props

Props stands for **Properties**.

Props are used to pass data from a Parent Component to a Child Component.

Flow:

Parent

↓

Child

Props are **read-only**. A child component should not modify the props it receives.

Passing Props

Child Component

```
function Student(props){  
  
  return (  
  
    <h1>{props.name}</h1>  
  
  );  
  
}  
  
export default Student;
```

Parent Component

```
import Student from './Student';
```

```
function App(){  
  
  return(  
  
    <div>  
  
      <Student name="Alice"/>  
  
      <Student name="Bob"/>  
  
    </div>  
  
  );  
  
}  
  
export default App;
```

Output

Alice

Bob

Multiple Props

```
function Student(props){  
  
  return(  
  
    <div>  
  
      <h2>Name : {props.name}</h2>  
  
      <p>Age : {props.age}</p>  
  
    </div>  
  
  );  
  
}
```

Calling

<Student

```
name="Alice"
```

```
age={20}
```

```
/>
```

Output

Name : Alice

Age : 20

Destructuring Props

Instead of

```
props.name
```

```
props.age
```

Use

```
function Student({name, age}){
```

```
  return(
```

```
    <div>
```

```
      <h1>{name}</h1>
```

```
      <p>{age}</p>
```

```
    </div>
```

```
  );
```

```
}
```

Cleaner and easier to read.

7. Events

Events allow React to respond to user actions.

Examples:

- Click
- Double Click
- Hover
- Submit
- Change
- Key Press

onClick Event

```
function App(){  
  
  function showMessage(){  
  
    alert("Button Clicked");  
  
  }  
  
  return(  
  
    <button onClick={showMessage}>  
  
      Click  
  
    </button>  
  
  );  
  
}
```

Arrow Function

```
<button  
  
  onClick={() => alert("Hello")}  
  
>  
  
Click  
  
</button>
```

onMouseOver

```
function App(){
```

```
return(  
  
<h1  
  
onMouseOver={() => alert("Mouse Entered")}  
  
>  
  
Hover Here  
  
</h1>  
  
);  
  
}
```

```
onChange  
function App(){  
  
function display(e){  
  
console.log(e.target.value);  
  
}  
  
return(  
  
<input  
  
onChange={display}  
  
/>  
  
);  
  
}
```

Typing

A

Console

A

Typing

AB

Console

AB

8. State

State stores data that changes over time. Unlike props, state belongs to the component itself and can be updated. Whenever state changes, React automatically re-renders the component.

To create state, React provides the `useState` Hook.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

Initially:

count = 0

After clicking the button:

count = 1
count = 2
count = 3

React updates the displayed value automatically.

State with Objects

Objects are commonly used when storing related information together.

```
import { useState } from "react";

function Student() {
  const [student, setStudent] = useState({
    name: "Alice",
    age: 20,
    city: "Mumbai"
  });

  function changeCity() {
    setStudent({
      ...student,
      city: "Delhi"
    });
  }

  return (
    <div>
      <h2>{student.name}</h2>
      <p>{student.age}</p>
      <p>{student.city}</p>

      <button onClick={changeCity}>
        Change City
      </button>
    </div>
  );
}

export default Student;
```

The spread operator (...student) copies the existing object. Without it, updating one property would remove the others.

State with Arrays

Arrays are useful for lists such as products, students, or tasks.

```
import { useState } from "react";

function Fruits() {
  const [fruits, setFruits] = useState([
```

```

    "Apple",
    "Mango",
    "Banana"
  ]);

  function addFruit() {
    setFruits([...fruits, "Orange"]);
  }

  return (
    <div>
      <ul>
        {fruits.map((fruit, index) => (
          <li key={index}>{fruit}</li>
        ))}
      </ul>

      <button onClick={addFruit}>
        Add Fruit
      </button>
    </div>
  );
}

```

export default Fruits;

Output initially:

Apple
Mango
Banana

After clicking the button:

Apple
Mango
Banana
Orange

React state should be treated as immutable. Instead of modifying the existing array using methods like `push()`, create a new array using the spread operator.

9. Forms

Forms collect input from users, such as login details, registration information, or feedback. In React, form inputs are commonly implemented as **controlled components**, where the input value is controlled by state.

Example: Text Input

```
import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        placeholder="Enter Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <h2>Hello, {name}</h2>
    </div>
  );
}

export default App;
```

As the user types, the state updates and the greeting changes automatically.

Handling Form Submission

```
import { useState } from "react";

function Login() {
  const [email, setEmail] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    alert(`Email: ${email}`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        placeholder="Enter Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
    </form>
  );
}
```

```

        <button type="submit">
          Submit
        </button>
      </form>
    );
  }

```

```
export default Login;
```

event.preventDefault() prevents the browser from refreshing the page when the form is submitted.

Handling Multiple Inputs

```
import { useState } from "react";
```

```
function Register() {
  const [form, setForm] = useState({
    name: "",
    email: ""
  });

```

```

  function handleChange(e) {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  }

```

```

  return (
    <div>
      <input
        name="name"
        placeholder="Name"
        value={form.name}
        onChange={handleChange}
      />

```

```

      <input
        name="email"
        placeholder="Email"
        value={form.email}
        onChange={handleChange}
      />

```

```

      <h3>Name: {form.name}</h3>
      <h3>Email: {form.email}</h3>

```

```
    </div>
  );
}
```

```
export default Register;
```

Using a single `handleChange` function makes it easy to manage forms with many input fields.

Summary

By the end of these topics, you should understand:

- Why React is used and how it differs from traditional JavaScript applications.
- How to set up a React project using Vite.
- How JSX combines HTML-like syntax with JavaScript.
- How components help build reusable and maintainable user interfaces.
- How props pass data from parent to child components.
- How events allow applications to respond to user interactions.
- How state manages dynamic data, including primitive values, objects, and arrays.
- How controlled forms use state to manage user input.

These concepts form the foundation of React. Once you are comfortable with them, the next logical topics are **Conditional Rendering**, **Lists and Keys**, **State Lifting**, **Prop Drilling**, **Hooks** (`useEffect`, `useRef`, `useMemo`, `useCallback`), **Custom Hooks**, and **React Router**.

My Card Code


```

                                <div className = "text-[12px]
text-zinc-500">Likes</div>
                                </div>
                                <div className = "text-center">
                                    <div className = "text-[15px] font-semibold
leading-none">828</div>
                                    <div className = "text-[12px]
text-zinc-500">Posts</div>
                                    </div>
                                    <div className = "text-center">
                                        <div className = "text-[15px] font-semibold
leading-none">342.9K</div>
                                        <div className = "text-[12px]
text-zinc-500">Views</div>
                                        </div>
                                    </div>
                                    <div className = "flex items-center justify-around
mt-5 gap-8">
                                        <div>
                                            <svg xmlns="http://www.w3.org/2000/svg"
width="22" height="22" fill="currentColor" viewBox="0 0 24 24" >
                                                <path d="M11.999 7.377a4.623 4.623 0 1 0 0
9.248 4.623 4.623 0 0 0 0-9.248m0 7.627a3.004 3.004 0 1 1 0-6.008 3.004
3.004 0 0 1 0 6.008M16.806 6.129a1.078 1.078 0 1 0 0 2.156 1.078 1.078
0 1 0 0-2.156"/><path d="M20.533 6.111A4.6 4.6 0 0 0 17.9 3.479a6.6 6.6
0 0 0-2.186-.42c-.963-.042-1.268-.054-3.71-.054s-2.755 0-3.71.054a6.6
6.6 0 0 0-2.184.42 4.6 4.6 0 0 0-2.633 2.632 6.6 6.6 0 0 0-.419
2.186c-.043.962-.056 1.267-.056 3.71s0 2.753.056 3.71c.015.748.156
1.486.419 2.187a4.6 4.6 0 0 0 2.634 2.632 6.6 6.6 0 0 0
2.185.45c.963.042 1.268.055 3.71.055s2.755 0 3.71-.055a6.6 6.6 0 0 0
2.186-.419 4.61 4.61 0 0 0
2.633-2.633c.263-.7404-1.438.419-2.186.043-.962.056-1.267.056-3.71s0-2
.753-.056-3.71a6.6 6.6 0 0 0-.421-2.217m-1.218 9.532a5 5 0 0 1-.311
1.688 2.99 2.99 0 0 1-1.712 1.711 5 5 0 0
1-1.67.311c-.95.044-1.218.055-3.654.055-2.438 0-2.687 0-3.655-.055a5 5
0 0 1-1.669-.311 2.99 2.99 0 0 1-1.719-1.711 5.1 5.1 0 0
1-.311-1.669c-.043-.95-.053-1.218-.053-3.654s0-2.686.053-3.655a5 5 0 0
1 .311-1.687c.305-.789.93-1.41 1.719-1.712a5 5 0 0 1
1.669-.311c.951-.043 1.218-.055 3.655-.055s2.687 0 3.654.055a5 5 0 0 1
1.67.311 3 3 0 0 1 1.712 1.712 5.1 5.1 0 0 1 .311 1.669c.043.951.054
1.218.054 3.655s0 2.698-.043 3.654z"/>
                                            </svg>
                                        </div>

```

```

        <div>
            <svg xmlns="http://www.w3.org/2000/svg"
width="22" height="22" fill="currentColor" viewBox="0 0 24 24" >
                <path d="M13.68 10.62 20.24 3h-1.55L13
9.62 8.45 3H3.19l6.88 10.01L3.19 21h1.55l6.01-6.99 4.8
6.99h5.24l-7.13-10.38Zm-2.13 2.47-.7-1-5.54-7.93H7.7l4.47 6.47 1 5.82
8.32H16.3z"></path>
            </svg>
        </div>
        <div>
            <svg xmlns="http://www.w3.org/2000/svg"
width="22" height="22" fill="currentColor" viewBox="0 0 24 24" >
                <path d="M16.39
11.27c-.09-.04-.17-.08-.26-.12-.15-.284-1.71-4.47-4.32-4.49h-.04c-1.56
0-2.86.67-3.66 1.88l1.44.98c.6-.91 1.53-1.1 2.22-1.1h.02c.86 0 1.51.26
1.93.74.31.35.51.84.61 1.46-.76-.13-1.59-.17-2.47-.12-2.48.14-4.08
1.59-3.97 3.6.05 1.02.56 1.9 1.43 2.47.73.48 1.68.72 2.66.67 1.3-.07
2.32-.57 3.03-1.47.54-.69.88-1.58 1.03-2.7.62.37 1.08.86 1.33 1.45.43 1
.46 2.65-.89 4-1.18 1.18-2.6 1.69-4.74
1.7-2.38-.02-4.17-.78-5.34-2.26-1.09-1.39-1.66-3.4-1.68-5.97.02-2.57.59
-4.58 1.68-5.97 1.17-1.49 2.97-2.25 5.34-2.26 2.39.02 4.22.78 5.43
2.28.59.73 1.04 1.65 1.34
2.73l1.68-.45c-.36-1.32-.92-2.46-1.69-3.4-1.56-1.91-3.83-2.89-6.76-2.91
h-.01c-2.92.02-5.17 1-6.68 2.92C3.71 6.64 3.01 9.02 2.99 12c.02 3 .72
5.37 2.06 7.08C6.56 21 8.81 21.98 11.73 22h.01c2.6-.02 4.43-.7
5.94-2.21 1.98-1.97 1.92-4.45
1.26-5.97-.47-1.09-1.36-1.97-2.58-2.56Zm-4.49
4.22c-1.09.06-2.22-.43-2.27-1.47-.04-.78.55-1.64
2.34-1.74.2-.01.41-.02.6-.02.65 0 1.26.06 1.81.18-.21 2.57-1.41
2.99-2.48 3.05"/>
            </svg>
        </div>
    </div>
</div>
);
}

export default App;

```